

Database Fundamentals

Contents

The relational model

1. [What a database is](#)
2. [Tables, rows and relationships](#)
3. [Keys](#)
4. [Constraints](#)
5. [SQL basics](#)
6. [Joins](#)
7. [NULL and three-valued logic](#)

Data modelling

8. [Normalization](#)
9. [When to denormalize](#)

How data is found

10. [How rows are stored](#)
11. [Indexes](#)
12. [Composite indexes and column order](#)
13. [Reading a query plan](#)

Correctness under concurrency

14. [Transactions and isolation](#)
15. [MVCC, Multi-Version Concurrency Control](#)
16. [Locking and deadlocks](#)

Scaling out

17. [Replication](#)
18. [Partitioning and sharding](#)
19. [SQL, NoSQL and CAP](#)

From Java

20. [Connection pooling](#)
21. [The N+1 problem](#)

Reference

- 22. [Interview questions](#)
- 23. [Common mistakes](#)
- 24. [Quick reference](#)

1. What a database is

Strictly, a **database** is an organised collection of data, and a **DBMS, Database Management System**, is the software that stores it, retrieves it and manages access to it. Postgres and MySQL are database management systems. In ordinary conversation people say "database" for both, and that is fine as long as the distinction is available when an interviewer asks for it.

What the management system gives us can be described as everything a plain file cannot do safely:

- **Concurrency.** Many clients read and write at the same time, and the result stays correct.
- **Durability.** Once a write is confirmed, a crash does not lose it.
- **Declarative querying.** We describe the rows we want, and the system decides how to find them.
- **Constraints.** Rules such as "an order must belong to a real customer" are enforced by the engine, not by hope.
- **Indexes.** Finding one row among fifty million does not mean reading fifty million rows.
- **Recovery.** After a crash, committed work is replayed and unfinished work is undone.

The cost is worth naming: it is another process to run, size, monitor, back up and upgrade, and it becomes the hardest part of a system to scale, because it is the part holding state.

2. Tables, rows and relationships

A **table** is a set of rows with the same columns. A **row** holds one fact about one thing. A **column** has a name and a type, and the type is a promise the engine enforces. The **schema** is the whole shape: tables, columns, types and constraints.

Relationships between tables come in three shapes, and being able to map each one is a common early question.

Relationship	How it is stored	Example
One to one	A foreign key on either side, with a unique constraint	A user and their single profile row
One to many	A foreign key on the many side	A customer has many orders, so <code>orders.customer_id</code>

Relationship	How it is stored	Example
Many to many	A third table holding two foreign keys	Students and courses, joined by <code>enrolment</code>

The rule worth memorizing for the one-to-many case: the foreign key always lives on the side that has many rows. An order points at its customer; a customer does not hold a list of order identifiers.

In Java that maps onto Jakarta Persistence API annotations:

```
@Entity
public class Order {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @ManyToOne(fetch = FetchType.LAZY)           // the FK column lives here
    @JoinColumn(name = "customer_id")
    private Customer customer;
}
```

`fetch = FetchType.LAZY` on a `@ManyToOne` is worth setting deliberately, because the default for that annotation is eager, and eager loading is one half of the problem in section 21.

3. Keys

Primary key. The column or columns identifying a row. It is unique and never null. A table has at most one primary key constraint, and that constraint may cover several columns. Most engines create an index for it automatically.

Composite key. A primary key made of more than one column, common in join tables where the pair of foreign keys is the identity.

Candidate key. Any column set that *could* serve as the primary key. The one chosen becomes the primary key, and the rest usually become unique constraints.

Foreign key. A column or column set referencing a candidate key in another table, which in practice is almost always the primary key but may be any unique constraint. It buys referential integrity: the engine refuses to store an order for a customer who does not exist, and refuses to delete a customer who still has orders unless a cascade rule says otherwise.

Unique constraint. Uniqueness without identity. A user table can have a surrogate primary key and still declare the email unique.

The design question that actually gets asked is **natural against surrogate**.

A *natural key* is data that already means something: an email address, an ISBN, a national insurance number. A *surrogate key* is a value generated purely to be an identifier, with no meaning outside the database.

Prefer surrogate keys. Natural keys change, and a primary key that changes has to be updated in every table referencing it. People change email addresses, countries reissue identifiers, and a value that is unique today is discovered to be non-unique later. Declare the natural value as a unique constraint instead, which keeps the guarantee without making it the identity.

Then the follow-up: **auto-increment or UUID, Universally Unique Identifier?**

	Auto-increment	Random UUID, version 4
Size	4 or 8 bytes	16 bytes, and larger again as text
Insert position	Always at the end of the index	Scattered, so pages split and cache hit rates fall
Generated by	The database, on insert	The application, before insert
Leaks information	Row counts and creation order	Nothing
Across shards or systems	Needs coordination	Safe without coordination

The middle row is the one people miss. A random identifier writes into a different part of the index every time, which fragments it and pushes more of it out of memory. Time-ordered identifiers such as UUID version 7 keep the ordering benefit of a sequence while staying generateable by the application, which is why they have become the usual recommendation when a UUID is needed.

4. Constraints

A constraint is a rule the engine enforces on every write, whatever the source.

Constraint	What it guarantees
PRIMARY KEY	Unique and not null, and identifies the row
FOREIGN KEY	References point at rows that exist, subject to the configured delete and update action
UNIQUE	No two rows share this value
NOT NULL	The column always holds a value
CHECK	A condition holds for every row
DEFAULT	A value used when the insert supplies none

```
CREATE TABLE customers (
  id          BIGSERIAL      PRIMARY KEY,
  email       VARCHAR(255)   NOT NULL UNIQUE,
  country     CHAR(2)        NOT NULL,
  credit      NUMERIC(12,2)  NOT NULL DEFAULT 0 CHECK (credit >= 0),
  created_at  TIMESTAMPTZ    NOT NULL DEFAULT now()
);

CREATE TABLE orders (
  id          BIGSERIAL      PRIMARY KEY,
  customer_id BIGINT         NOT NULL REFERENCES customers(id) ON DELETE RESTRICT,
  total       NUMERIC(12,2)  NOT NULL CHECK (total > 0)
);
```

A foreign key also decides what happens when the referenced row is deleted. `ON DELETE RESTRICT` refuses the delete, `CASCADE` deletes the children too, and `SET NULL` clears the reference. `CASCADE` is convenient and worth using carefully, because one delete can remove far more than expected.

The principle underneath all of this: **enforce an invariant as close to the data as practical**. Application validation gives better error messages and catches problems earlier, so it is worth having. It is not a guarantee, because the application is never the only writer. Migrations, scripts, another service, and one engineer with a database console all write to the same tables, and only the constraint stops them.

In a Spring application that means both, not either:

```
public record CreateCustomer(
    @NotBlank @Email String email,      // rejected early, with a good message
    @NotBlank String country) { }
```

The annotation improves the error the caller sees. The `NOT NULL UNIQUE` on the column is what makes the rule true.

5. SQL basics

SQL, Structured Query Language, is declarative. We describe the rows we want and the engine decides how to get them, which is why section 13 on query plans matters.

A query with every common clause:

```
SELECT customer_id, count(*) AS order_count
FROM orders
WHERE status = 'completed'
GROUP BY customer_id
HAVING count(*) > 5
ORDER BY order_count DESC
LIMIT 20;
```

The clauses are **written** in that order but **evaluated** in a different one, and knowing the difference answers several interview questions at once:

FROM → WHERE → GROUP BY → HAVING → SELECT → ORDER BY → LIMIT

Two consequences fall straight out of it. An alias created in `SELECT` cannot be used in `WHERE`, because `WHERE` runs first, but it can be used in `ORDER BY`, because that runs later. And `WHERE` cannot reference an aggregate, because grouping has not happened yet.

That last point is the whole reason `HAVING` exists:

Clause	Filters	Can use aggregates
<code>WHERE</code>	Individual rows, before grouping	No
<code>HAVING</code>	Groups, after grouping	Yes

The aggregate functions are `COUNT`, `SUM`, `AVG`, `MIN` and `MAX`. In portable SQL, every column in the `SELECT` list that is not inside an aggregate should appear in `GROUP BY`. Some engines relax that where the column is functionally dependent on what is already grouped, so Postgres allows selecting a table's other columns when the grouping is by that table's primary key, because the key determines them.

`DISTINCT` removes duplicate rows from the result, and it applies to the whole selected row rather than one column. It is not free, because the engine has to sort or hash to do it, and using it to paper over a join that produces duplicates hides the real problem.

The write statements:

```
INSERT INTO customers (email, country) VALUES ('a@b.com', 'GB');

UPDATE orders SET status = 'shipped' WHERE id = 12345;

DELETE FROM orders WHERE created_at < '2024-01-01';
```

Always name the columns in an `INSERT` rather than relying on table order, and always include a `WHERE` in an `UPDATE` or `DELETE` unless every row really is the target.

6. Joins

A join combines rows from two tables using a condition, almost always a foreign key matching a primary key.

Join	Returns
INNER JOIN	Only rows matching on both sides
LEFT JOIN	Every row from the left, with nulls where the right has no match
RIGHT JOIN	The mirror image, and rare, because people flip the tables instead
FULL OUTER JOIN	Every row from both sides, with nulls where either has no match
CROSS JOIN	Every combination of both sides, so a Cartesian product
Self join	A table joined to itself, using aliases

```
-- Every customer, with their orders where any exist
SELECT c.name, o.total
FROM customers c
LEFT JOIN orders o ON o.customer_id = c.id;
```

A self join answers questions inside one table, such as each employee beside their manager:

```
SELECT e.name AS employee, m.name AS manager
FROM employees e
LEFT JOIN employees m ON m.id = e.manager_id;
```

The one join mistake worth memorizing

A condition on the right table placed in `WHERE` silently turns a `LEFT JOIN` into an `INNER JOIN`, because the unmatched rows hold `NULL` there and `NULL` fails every comparison.

```
-- Looks like a LEFT JOIN, behaves like an INNER JOIN
SELECT c.name, o.total
FROM customers c
LEFT JOIN orders o ON o.customer_id = c.id
WHERE o.status = 'completed';           -- drops every customer with no orders

-- Fixed: the filter belongs in the join condition
SELECT c.name, o.total
FROM customers c
LEFT JOIN orders o ON o.customer_id = c.id AND o.status = 'completed';
```

The exception is deliberate, and it is the standard way to find rows with no match at all:

```
-- Customers who have never ordered. This is the anti-join idiom.
SELECT c.name
FROM customers c
LEFT JOIN orders o ON o.customer_id = c.id
WHERE o.id IS NULL;
```

How a join is actually executed, whether by nested loop, hash or merge, is the engine's decision and shows up in the plan. Section 13 covers reading it.

7. NULL and three-valued logic

`NULL` means the value is unknown. It is not zero, not an empty string, and not false. Because it is unknown, comparing anything to it produces neither true nor false but a third result, **unknown**, and SQL evaluates conditions in that three-valued logic.

```
SELECT NULL = NULL;           -- NULL, not true
SELECT NULL = 0;              -- NULL
SELECT NULL <> 'anything';    -- NULL
```

A row is only returned when the condition evaluates to true, so unknown behaves like false for filtering. That produces the most common mistake in SQL:

```
SELECT * FROM customers WHERE deleted_at = NULL;    -- always zero rows
SELECT * FROM customers WHERE deleted_at IS NULL;    -- correct
```

Negation does not rescue it either. `NOT (NULL = 1)` is still unknown, so the row still fails the filter.

NOT IN with a null in the list is the trap that catches experienced people:

```
-- Returns nothing at all, whatever the data, if the subquery yields any NULL
SELECT * FROM customers
WHERE id NOT IN (SELECT customer_id FROM orders);
```

If any `customer_id` is null, the comparison becomes unknown for every row. `NOT EXISTS` or the anti-join from section 6 avoids it entirely.

Aggregates skip nulls, except `COUNT(*)`:

Expression	Behavior
<code>COUNT(*)</code>	Counts rows, including rows that are entirely null
<code>COUNT(col)</code>	Counts rows where <code>col</code> is not null

Expression	Behavior
<code>COUNT(DISTINCT col)</code>	Counts distinct non-null values
<code>SUM</code> , <code>AVG</code> , <code>MIN</code> , <code>MAX</code>	Ignore nulls entirely

So `AVG(price)` over five rows where one price is null divides by four, not five. That is usually what we want and occasionally a serious reporting bug.

Uniqueness and nulls are engine-specific. Postgres and MySQL allow many nulls in a unique column, because two unknown values are not known to be equal. SQL Server allows only one. Naming that difference rather than asserting one behavior is the safer answer.

`COALESCE` returns the first non-null argument, and it earns its place after a left join where the unmatched side comes back null:

```
SELECT c.name, COALESCE(SUM(o.total), 0) AS lifetime_value
FROM customers c
LEFT JOIN orders o ON o.customer_id = c.id
GROUP BY c.id, c.name;
```

In Java, `ResultSet.getInt` returns `0` for a database null, which is indistinguishable from a real zero unless `wasNull()` is checked immediately afterwards. Mapping the column to `Integer` rather than `int` avoids the whole class of bug.

8. Normalization

Normalization is the process of arranging columns so that each fact is stored once. The goal is to remove the update anomalies that duplication causes: the same value changed in one row and missed in another.

Three normal forms cover almost every interview.

1NF, First Normal Form. Every column holds a single value, and there are no repeating groups.

```
BAD  orders(id, customer, items)
      1, "Ana", "shoes, hat, socks"    -- three values in one column

GOOD orders(id, customer)
      order_items(order_id, item)      -- one row per item
```

2NF, Second Normal Form. In 1NF, and no non-key column depends on only part of a composite primary key. This only applies when the primary key has more than one column.

```
BAD  order_items(order_id, product_id, quantity, product_name)
      -- primary key is (order_id, product_id)
      -- product_name depends on product_id alone, not on the pair

GOOD order_items(order_id, product_id, quantity)
      products(product_id, product_name)
```

3NF, Third Normal Form. In 2NF, and no non-key column depends on another non-key column.

```
BAD  orders(id, customer_id, customer_email)
      -- customer_email depends on customer_id, which is not the key

GOOD orders(id, customer_id)
      customers(id, email)
```

The informal summary that works well when speaking: every non-key column depends on the key, the whole key, and nothing but the key.

BCNF, Boyce Codd Normal Form, is a stricter version of 3NF that matters when a table has several overlapping candidate keys. Worth naming, rarely worth more in an interview.

9. When to denormalize

Denormalizing means deliberately storing a fact twice to make reads cheaper. It is a real technique, not a failure, and the interview answer is about when the trade pays.

Reasonable cases:

- A total that is expensive to compute and read constantly, such as an order total stored on the order rather than summed from its lines every time.
- A reporting or analytics table built for one query shape, refreshed on a schedule.
- A join across several large tables sitting on the hottest path in the application.

The cost is always the same: two copies of one fact can disagree. Something has to keep them in step, whether that is application code, a trigger, or a **materialized view**, which is a query whose results the database stores and refreshes on demand. A plain **view** is different: it is a stored query name with no stored data, evaluated every time it is used, so it simplifies code without changing performance.

The order that survives scrutiny: normalize first, measure, then denormalize the specific thing that is slow, with a written plan for keeping the copy correct. Denormalizing on a guess produces duplication and no speed.

10. How rows are stored

Rows live in fixed-size blocks called **pages**, and the page is the unit the engine reads and writes. Postgres uses 8KB by default and MySQL InnoDB uses 16KB. Reading one row means reading its whole page, which is why row size and how rows are packed affect performance.

The two major engines organize tables differently, and the difference explains several behaviors:

MySQL InnoDB uses a clustered index. The primary key *is* the table. Rows are stored inside the primary key's B-tree, in key order. A secondary index does not store a row pointer; it stores the primary key value. So a lookup through a secondary index is two steps: find the primary key in the secondary index, then find the row in the clustered index. This is why a large primary key makes every secondary index larger, and why primary key order affects insert cost.

Postgres uses a heap. The table is an unordered collection of pages, and every index, including the primary key's, holds a pointer to a physical row location. All indexes are equal, and there is no second lookup step. The trade is that Postgres keeps old row versions in the heap, which section 15 covers.

Neither is better. The point is that a fact such as "the primary key affects the size of every other index" is true of one and not the other, so the honest answer names the engine.

11. Indexes

An index is a separate, ordered structure that lets the engine find rows without reading the whole table. The default type nearly everywhere is a **B-tree**, and the implementation is usually a B+tree, where the internal nodes hold only keys and the leaves hold the entries in sorted order and are linked to each other.

That shape is what an index can do:

- Equality lookups, `WHERE id = 42`
- Range scans, `WHERE created_at > ?`, because the leaves are in order
- Prefix matching, `LIKE 'abc%'`
- Sorting, since reading the leaves in order is already sorted
- Minimum and maximum, by reading one end

What an index costs. Every insert, update and delete that touches an indexed column has to update the index as well, so write throughput falls with each index added. Indexes take disk and memory. And an index nobody uses costs all of that and returns nothing, which is why unused indexes are worth hunting for.

When an index does not help, which is the follow-up question that separates people:

- **Low selectivity.** An index on a boolean column that splits rows evenly is usually ignored, because reading half the table through an index is slower than reading it directly.
- **A leading wildcard.** `LIKE '%abc'` cannot use a B-tree, because the tree is ordered from the left.

- **A function on the column.** `WHERE lower(email) = ?` cannot use a plain index on `email`. The fix is an index on the expression itself.
- **Implicit type conversion.** Comparing a `varchar` column against a number forces a conversion, and which side gets converted depends on the engine and the types involved. When it is the column, the effect is the same as wrapping it in a function and the index becomes unusable.
- **Stale statistics.** The planner chooses using estimates. If the estimates are wrong it makes bad choices, which is what `ANALYZE` fixes.

Other index types worth naming. A **hash** index serves equality only and cannot help ranges or sorting. A **partial** index covers only the rows matching a condition, which is efficient for something like the small set of unprocessed rows in a large queue table. Postgres **GIN** indexes serve containment queries inside arrays, `JSONB` and full text.

A covering index contains every column a query needs, so the engine can answer it without fetching the rows themselves. Postgres calls that an index-only scan, and allows it only when the visibility information also permits it, so a page not yet marked all-visible still sends the engine to the table.

12. Composite indexes and column order

An index over several columns is ordered by the first column, then the second within that, and so on, exactly like sorting a list by several keys. That ordering produces the **leftmost prefix rule**, which is one of the most reliably asked index questions.

Given an index on `(customer_id, status, created_at)`:

Query filters on	Uses the index
<code>customer_id</code>	Yes
<code>customer_id, status</code>	Yes
<code>customer_id, status, created_at</code>	Yes, fully
<code>status</code> alone	No useful seek
<code>status, created_at</code>	No useful seek
<code>customer_id, created_at</code>	Partly, seeking on <code>customer_id</code> then filtering

Postgres can sometimes still read the whole index for a non-leading column, which is cheaper than reading the table but far from a seek. The rule holds as guidance in both engines.

Ordering the columns well:

1. Columns compared with equality go first.
2. A column compared with a range goes after those, because everything after a range column loses its ordering usefulness.

3. Columns needed only for sorting go last.

One practical consequence: an index on `(a, b)` can often make a separate index on `a` unnecessary, since the composite serves the same lookups. It is not automatically redundant, because the narrower index is smaller, cheaper to scan and more likely to stay in memory, so check the plans and the workload before dropping it. The reverse never holds: an index on `a` cannot do the work of `(a, b)`.

13. Reading a query plan

`EXPLAIN` shows the plan the engine intends to use. `EXPLAIN ANALYZE` runs the query and shows what actually happened, so it is the one worth using, with the warning that it really does execute the statement.

What to look at, in order:

- **Scan type.** A sequential scan on a large table with a selective filter usually means an index is missing or unusable. A sequential scan on a small table is fine and often faster than an index.
- **Estimated against actual rows.** A large gap means the statistics are stale, and every decision built on that estimate is suspect.
- **Join strategy.** A nested loop is right when one side is tiny. A hash join suits two large unsorted inputs. A merge join suits inputs already sorted, often because an index provided the order.
- **Rows removed by filter.** A large number means the engine read rows it then threw away, which is the signature of an index that gets the query close but not close enough.

The instinct to build: a slow query is a question about the plan, not a reason to add an index and hope.

14. Transactions and isolation

A **transaction** is a group of statements the database treats as one unit of work, and **ACID** is the four promises it makes about that unit: **A**tomicity, **C**onsistency, **I**solation and **D**urability.

Isolation is the one with settings, because full isolation costs concurrency. Each weaker level allows specific problems, called anomalies, and the four worth being able to describe are these.

Dirty read. Reading a row another transaction has written but not committed. If that transaction rolls back, we acted on a value that never existed.

```
T1: UPDATE accounts SET balance = 50 WHERE id = 1;    -- not committed
T2: SELECT balance FROM accounts WHERE id = 1;        -- reads 50
T1: ROLLBACK;                                         -- the 50 never existed
```

Non-repeatable read. The same row read twice in one transaction gives different values, because another transaction committed a change in between.

```
T1: SELECT balance FROM accounts WHERE id = 1;      -- 100
T2: UPDATE accounts SET balance = 50 WHERE id = 1; COMMIT;
T1: SELECT balance FROM accounts WHERE id = 1;      -- 50, same transaction
```

Phantom read. The same query run twice returns a different set of rows, because another transaction inserted or deleted rows matching the filter.

```
T1: SELECT count(*) FROM orders WHERE total > 100;  -- 12
T2: INSERT INTO orders (total) VALUES (500); COMMIT;
T1: SELECT count(*) FROM orders WHERE total > 100;  -- 13
```

Lost update. Two transactions read the same row, both compute a new value, and the second write overwrites the first.

```
T1: SELECT balance FROM accounts WHERE id = 1;      -- 100
T2: SELECT balance FROM accounts WHERE id = 1;      -- 100
T1: UPDATE accounts SET balance = 90 WHERE id = 1; COMMIT;
T2: UPDATE accounts SET balance = 80 WHERE id = 1; COMMIT;  -- T1's change gone
```

The standard levels, and which of the first three each one blocks:

Level	Dirty read	Non-repeatable read	Phantom read
Read uncommitted	Possible	Possible	Possible
Read committed	Blocked	Possible	Possible
Repeatable read	Blocked	Blocked	Possible
Serializable	Blocked	Blocked	Blocked

Postgres defaults to read committed and MySQL InnoDB defaults to repeatable read. The table is the standard's minimum rather than a description: an engine may be stricter, and Postgres repeatable read blocks phantom reads as well, which the standard does not require.

One anomaly sits outside that table. **Write skew** is where two transactions each read an overlapping set of rows, each check a rule that currently holds, and each write a different row, so that together they break a rule neither broke alone. Snapshot isolation permits it, and only serializable prevents it. The companion ACID document works through it, along with what each engine does and how transaction boundaries behave in Spring.

15. MVCC, Multi-Version Concurrency Control

Locking readers out while a writer works would be correct and slow. **MVCC** avoids it by keeping more than one version of a row, and giving each transaction a consistent view of the data as it stood when that transaction started reading.

The result is the property worth stating in an interview: **for ordinary reads, readers do not block writers and writers do not block readers**. Two writers touching the same row still conflict. Explicit locking sits deliberately outside this, since `SELECT ... FOR UPDATE` exists precisely so that a reader does block writers, and some schema changes take a lock that blocks everything.

The engines store the old versions differently, and that difference explains real operational behavior:

Postgres keeps old versions in the table. An `UPDATE` writes a new row version and marks the previous one dead rather than overwriting it. A dead version cannot be removed while any transaction might still need to see it, so `VACUUM` reclaims them later. Two consequences follow directly: a table physically grows with dead rows until vacuumed, which is called bloat, and a long-running transaction holds back cleanup for the whole database because its snapshot might still need those versions. That is the mechanism behind the advice to keep transactions short.

MySQL InnoDB keeps old versions in the undo log, separately from the table, and purges them once no transaction needs them. The table itself stays compact, and the undo log is what grows under a long transaction instead.

MVCC is also what makes repeatable read cheap. The transaction reads its own snapshot and sees a stable picture without holding read locks on everything it touched.

16. Locking and deadlocks

Locks are how the engine stops two writers corrupting the same row. A **shared** lock allows other readers and blocks writers. An **exclusive** lock blocks both. Locks are usually taken on rows, though operations such as some schema changes take a table lock. Whether an engine ever converts many fine-grained locks into one coarser lock is engine-specific: SQL Server does that kind of escalation, while Postgres and InnoDB do not escalate row locks into table locks.

The choice that comes up in Java interviews is **pessimistic against optimistic** locking.

Pessimistic locking takes the lock before doing the work, so nobody else can interfere:

```
@Lock(LockModeType.PESSIMISTIC_WRITE)
@Query("select a from Account a where a.id = :id")
Optional<Account> findByIdForUpdate(@Param("id") Long id);
```

That becomes `SELECT ... FOR UPDATE`, and the lock is held until the transaction ends. It is correct under heavy contention and it costs concurrency, because other transactions wait.

Optimistic locking takes no explicit lock before the work. It records a version and checks at write time that nobody else changed the row:

```
@Entity
public class Account {

    @Id
    private Long id;

    @Version                                // JPA increments and checks this
    private int version;
}
```

The update becomes `... WHERE id = ? AND version = ?`. If it affects zero rows, someone else got there first and JPA raises `OptimisticLockException`, which Spring surfaces as `ObjectOptimisticLockingFailureException`. The caller retries. The `UPDATE` itself still takes an ordinary row lock while it runs; what is avoided is holding a lock across the read and the think time before it.

Optimistic locking suits low contention, which is most of the time. Pessimistic suits a small number of hot rows where retries would keep failing.

Deadlocks happen when two transactions hold locks the other needs. The engine detects the cycle, kills one transaction, and returns an error to that caller. That is normal operation rather than a fault, and the caller retries. The prevention is a consistent lock ordering: if every code path locks accounts in ascending identifier order, the cycle cannot form.

17. Replication

Replication copies data from one database to others. The usual arrangement is one **primary** taking writes and one or more **replicas** taking reads.

Asynchronous replication confirms the write on the primary and sends it onward afterwards. It is fast, and a failover can lose the most recent writes. **Synchronous** replication waits for a replica to confirm before the commit returns. It is safer and slower, and it makes the primary depend on the replica being healthy.

Two consequences that appear as real bugs:

Replication lag. A replica is behind by anything from milliseconds to minutes. Code that writes and then immediately reads from a replica can fail to see its own write, which users experience as a change that did not save. The fixes are routing a read to the primary after a write, or waiting for the replica to catch up.

Failover and split brain. When a primary fails, something must promote a replica. If the old primary comes back believing it is still primary, two nodes accept writes and the data diverges. This is why promotion is coordinated rather than automatic in simple setups.

Replication scales reads and gives redundancy. It does not scale writes, because every write still goes through one primary. That is what the next section is for.

18. Partitioning and sharding

Partitioning splits one table into pieces inside a single database. The engine knows about the pieces and can skip the irrelevant ones, which is called partition pruning. Common strategies are by range, such as one partition per month, by list, such as one per region, and by hash.

Sharding splits data across separate database servers, each holding a subset. The application or a routing layer decides which shard a query goes to.

The decision that matters most is the **shard key**, because it is hard to change afterwards:

- A key with uneven distribution creates a hot shard doing most of the work.
- A key that does not match how the data is queried forces queries to visit every shard and combine results.
- Joins and transactions spanning shards become significantly harder. Some distributed databases support them, and where the database does not, the application has to.

Sharding is the last option rather than the first. Before it: add indexes, fix the slow queries, add read replicas, add caching, move large cold data out, and consider a bigger machine. Sharding solves a write-throughput or data-size problem that none of those can, and it charges for it in operational complexity forever.

19. SQL, NoSQL and CAP

NoSQL is not one thing. It names four families with little in common beyond not being relational.

Family	Shape	Examples	Suits
Key-value	Value fetched by one key	Redis, DynamoDB	Caching, sessions, anything keyed
Document	Self-contained records, usually JSON	MongoDB	Records read and written whole
Wide-column	Rows with dynamic columns, grouped by partition	Cassandra, HBase	Very high write volume, time series
Graph	Nodes and edges as first-class objects	Neo4j	Relationship traversal, several hops deep

Choosing well is about access patterns rather than speed. A relational database suits data with many relationships, queries that are not known in advance, and rules the engine should enforce. A document store suits records fetched whole by identifier, where the schema varies. A graph database suits questions such as "who is connected to this person within three steps", which a relational database answers only with painful recursive queries.

The honest interview answer avoids both "NoSQL is faster" and "NoSQL means weaker guarantees". Individual systems differ enormously, and several now offer transactions, strong consistency and secondary indexes. The accurate framing is that each engine makes its own trade among schema flexibility, consistency, transaction scope, query capability and horizontal scale, so the question is always which engine and which trade.

CAP stands for **C**onsistency, **A**vailability and **P**artition tolerance. The theorem says that when a network partition splits the system, it must choose between staying consistent and staying available. Partitions are not optional in a distributed system, so the real choice is the other two.

Two precisions worth having ready. Consistency in CAP means **linearizability**: operations behave as though there were one real-time ordering of them, so a read reflects the latest completed write. That is not the C in ACID, where consistency means the declared constraints still hold. And CAP describes behavior *during a partition*, so no system sits permanently in one category.

20. Connection pooling

Opening a database connection is expensive: a TCP connection, authentication, and session setup, often tens of milliseconds. Doing that per request wastes most of the request. A **connection pool** opens a set of connections once and lends them out.

Spring Boot uses HikariCP by default:

```
spring.datasource.hikari.maximum-pool-size=10
spring.datasource.hikari.connection-timeout=3000
spring.datasource.hikari.leak-detection-threshold=20000
```

The sizing question is a good interview question because the intuitive answer is wrong. **More connections do not automatically mean more throughput.** The database can only do so much concurrent work, and past that point extra connections add context switching and lock contention instead. Once the database is saturated, a pool of ten can outperform a pool of a hundred. The right size depends on workload, query latency, cores and storage, so the HikariCP documentation offers `connections = ((core count * 2) + effective spindle count)` as a starting point to measure from rather than a formula to trust.

Three traps:

- **Leaked connections.** A connection that is never closed is never returned to the pool, so it stays unavailable to everything else. Enough of them and requests queue for a connection that is not coming back. `leak-detection-threshold` does not reclaim it; it logs a warning with a stack trace once a connection has been held longer than the threshold, which is what makes the leak findable.
- **Long transactions holding a connection.** A transaction that stays open across a call to an external service holds its connection for that whole time. Open transactions late and commit early.
- **Total connections across instances.** Pool size multiplied by the number of application instances must stay under the database's connection limit, and it is easy to exceed it after scaling out.

21. The N+1 problem

One of the most common database performance problems in Java applications, and a question worth expecting whenever an interview touches an ORM, Object Relational Mapping.

```
List<Order> orders = orderRepository.findAll();    // 1 query

for (Order order : orders) {
    System.out.println(order.getCustomer().getName());
    // one more query per order, because customer was lazy
}
```

One query fetches the orders. Then each access to a lazily loaded association fires its own query. A hundred orders means a hundred and one queries, each with its own round trip, and the page that was fast with ten rows of test data is unusable with real data.

The fix is to fetch what is needed in one query:

```
@Query("select o from Order o join fetch o.customer")
List<Order> findAllWithCustomer();
```

Or declaratively, which keeps the query method readable:

```
@EntityGraph(attributePaths = "customer")
List<Order> findAll();
```

Two details that make the answer stronger:

Eager fetching is not the fix. Changing the association to `FetchType.EAGER` stops these particular queries but makes every query that touches an order load its customer, whether or not the customer was wanted, and it hides the problem rather than solving it at the call site that has it.

Fetching a collection breaks pagination. `join fetch` on a `@OneToMany` returns one row per child, so the database cannot apply a row limit to the parents. Depending on version and configuration, Hibernate may warn and apply the paging in memory after loading everything, which can be far worse than the problem being fixed. For collections with pagination, use batch fetching with `@BatchSize` or fetch identifiers first and the children second.

22. Interview questions

Each answer below is written to be said out loud as it stands. The last sentence always leads toward the next question rather than closing the topic.

"What is the difference between WHERE and HAVING?"

`WHERE` filters individual rows before any grouping happens, and `HAVING` filters the groups afterwards. That follows from the order the clauses are evaluated in, which is `FROM`, then `WHERE`, then `GROUP BY`, then `HAVING`, then `SELECT`, then `ORDER BY`, then `LIMIT`. Because `WHERE` runs before grouping, it cannot reference an aggregate, which is the whole reason `HAVING` exists. The same evaluation order explains something people find surprising, which is that an alias defined in `SELECT` cannot be used in `WHERE` but can be used in `ORDER BY`, since one runs before the alias exists and the other after.

"What happens when you compare a column to NULL?"

The result is neither true nor false, it is unknown, because `NULL` means the value is not known rather than being zero or empty. SQL uses three-valued logic, and a row is only returned when the condition is actually true, so unknown behaves like false for filtering. That is why `WHERE deleted_at = NULL` returns nothing at all and `IS NULL` is the only correct form. The version that catches experienced people is `NOT IN` with a subquery that can produce a null, because one null makes the comparison unknown for every row and the whole query returns nothing, which is why I use `NOT EXISTS` or an anti-join instead. It also affects aggregates, since `COUNT(*)` counts rows but `COUNT(column)` and `AVG` skip nulls entirely.

"Why might a LEFT JOIN behave like an INNER JOIN?"

Because a condition on the right-hand table was put in the `WHERE` clause instead of the `ON` clause. The left join produces nulls for the unmatched rows, and any comparison against null is unknown, so those rows fail the filter and disappear, which leaves exactly the inner join result. The fix is to move the condition into the join condition, so it decides what counts as a match rather than filtering the result afterwards. The one deliberate exception is testing for `IS NULL` on the right-hand side, which is the standard anti-join idiom for finding rows with no match, such as customers who have never ordered.

"What is an index, and what does it cost?"

An index is a separate ordered structure, almost always a B-tree, that lets the engine find rows without reading the whole table. Because the leaves are held in sorted order, it serves equality lookups, range scans, prefix matches like `LIKE 'abc%'`, and sorting, since reading the index in order is already sorted. The cost is on writes: every insert, update or delete touching an indexed column has to maintain the index too, so each index makes writes slower and takes disk and memory. That is why an unused index is worse than no index, and why the interesting question is usually not "should we add one" but "which of these existing ones is earning its place".

"Why might the database ignore an index you added?"

Several reasons, and the first is selectivity. If the column does not narrow the result much, such as a boolean splitting rows evenly, reading half the table through an index is slower than just reading the table, and the planner is right to skip it. The others are usually the query's fault rather than the index's: a leading wildcard like `'%abc'` cannot use a B-tree because the tree is ordered from the left, a function around the column such as `lower(email)` means the index on the plain column does not apply, and an implicit type conversion does the same thing invisibly. The last possibility is stale statistics, where the planner's estimates are wrong, and running `ANALYZE` fixes that. The way to find out which one it is, rather than guessing, is `EXPLAIN ANALYZE`.

"Take an index on `customer_id`, `status` and `created_at`. Which queries use it?"

It is ordered by the first column, then the second within that, so it follows the leftmost prefix rule. A query filtering on `customer_id` uses it, so does one on `customer_id` and `status`, and so does one using all three. Filtering on `status` alone does not get a useful seek, because the index is not ordered by status at the top level. Filtering on `customer_id` and `created_at` seeks on the first column and then filters, so it helps partly. When I choose the order I put equality columns first, then any range column, then columns only needed for sorting, because everything after a range column loses its ordering benefit. A composite on `a` and `b` will often make a separate index on `a` unnecessary, though I would check the plans before dropping it, since the narrower index is smaller and may stay in memory better.

"What is MVCC?"

Multi-version concurrency control. Rather than locking readers out while a writer works, the engine keeps more than one version of each row and gives every transaction a consistent view of the data as it stood when that transaction began reading. The property that matters is that for ordinary reads, readers do not block writers and writers do not block readers, so the real conflict is two writers on the same row. That is about ordinary reads rather than everything, since `SELECT ... FOR UPDATE` is there precisely to block, and some schema changes lock out everyone. Where the old versions live differs by engine, and it explains real behaviour: Postgres keeps them in the table itself, so a table physically grows with dead rows until `VACUUM` reclaims them, and a long-running transaction holds that cleanup back because its snapshot might still need them. InnoDB keeps them in the undo log instead, so the table stays compact and the undo log grows. It is also what makes repeatable read cheap, since the transaction reads its own snapshot without holding read locks.

"Explain normalization up to third normal form."

First normal form means each column holds a single value, so no comma-separated list of items in one field. Second normal form applies when the primary key has several columns, and it means no column depends on only part of that key, so a product name does not belong in an order line table keyed by order and product, because the name depends on the product alone. Third normal form means no non-key column depends on another non-key column, so a customer email does not belong on the order row, because it depends on the customer rather than the order. The short version people say is that every non-key column depends on the key, the whole key, and nothing but the key. Beyond that there is Boyce Codd normal form, which tightens third normal form when a table has overlapping candidate keys.

"Auto-increment or UUID for a primary key?"

Auto-increment by default, because it is small, and because each insert lands at the end of the index rather than scattering writes across it, which keeps the index compact and cache-friendly. The reasons to want a UUID are that the application can generate it before the insert, that it does not leak row counts or creation order, and that it works across shards or systems without coordination. The problem with a random version 4 UUID is exactly the insert pattern: every write goes somewhere different in the index, so pages split and more of the index has to stay in memory. That is what UUID version 7 addresses by making it time ordered, which keeps the sequential behaviour while still being generated by the application, so that is what I would use when a UUID is genuinely needed.

"What is the N+1 problem and how would you fix it?"

It is when one query loads a list, and then each item triggers another query for a lazily loaded association, so a hundred orders become a hundred and one queries. It usually appears with an ORM, and it looks fine in testing because ten test rows hide it. The fix is to load what is needed in one statement, with a `join fetch` in the query or an `@EntityGraph` on the repository method. What I would avoid is switching the association to eager fetching, because that stops these queries and makes every other query load the association whether or not it was wanted. One case worth flagging is fetching a collection with pagination, because `join fetch` on a one-to-many returns a row per child so the limit cannot be applied in the database. Hibernate may then warn and page in memory instead, and for that case batch fetching is the better answer.

"SQL or NoSQL?"

It depends on the access pattern rather than on speed, and I would start by asking what the queries look like. Relational suits data with many relationships, queries not known in advance, and rules the engine should enforce, because the database does the joining and the constraint checking. NoSQL is really four different families: key-value for anything fetched by a single key, document for records read and written whole where the schema varies, wide-column for very high write volume such as time series, and graph for questions that traverse relationships several steps deep. So the answer is usually which one, not whether. What I would avoid saying is that NoSQL is faster, or that it always means weaker guarantees, because several of these engines now offer transactions and strong consistency, and each one makes its own trade between schema flexibility, consistency, query capability and horizontal scale.

"How do you size a connection pool?"

Smaller than people expect. The instinct is that more connections means more throughput, but the database can only do so much concurrent work, and past that point extra connections add context switching and lock contention instead. Once the database is saturated, a pool of ten can beat a pool of a hundred. The right number depends on the workload, the query latency, the cores and the storage, so HikariCP's documentation gives core count times two plus effective spindle count as a starting point to measure from rather than a rule. The two things I would check alongside it are whether connections are being held longer than needed, since a transaction left open across an external API call holds one for that whole time, and whether pool size multiplied by the number of instances still fits under the database's connection limit after scaling out.

23. Common mistakes

- Writing `= NULL` instead of `IS NULL`, which returns nothing and looks like missing data rather than a bug in the condition.
- Using `NOT IN` against a subquery that can produce a null, which makes the whole query return nothing regardless of the data.
- Filtering the right-hand table of a `LEFT JOIN` in the `WHERE` clause, which quietly converts it into an inner join.
- Relying on application validation alone for a rule that matters, when migrations, scripts and other services write to the same tables.
- Adding an index for every slow query without checking the plan. Some of those queries are slow for another reason, and each index makes every write slower.
- Putting a function around an indexed column in the `WHERE` clause, which quietly disables the index. `WHERE lower(email) = ?` needs an index on the expression.
- Using a natural key as the primary key, then discovering the value changes and has to be updated everywhere referencing it.
- Choosing a random UUID as a clustered primary key without knowing what it does to insert locality and index size.
- Denormalizing before measuring, which produces duplication and no speed improvement.
- Reading from a replica immediately after writing to the primary, then treating the missing row as a bug in the application rather than replication lag.
- Leaving a transaction open for a long time, which in Postgres holds back `VACUUM` for the whole database and lets dead rows accumulate.
- Leaving `FetchType.EAGER` on associations to avoid N+1. It moves the cost onto every other query instead.
- Sizing a connection pool by guessing upward, or scaling out instances without checking that total connections still fit under the database limit.

- Treating a deadlock as a fault rather than a normal outcome to retry, and paging someone about it.

24. Quick reference

Term	Meaning
DBMS	The software managing the data. The database is the data itself
Primary key	The column set identifying a row, unique and never null
Foreign key	A reference to a candidate key elsewhere, enforcing referential integrity
Surrogate key	A generated identifier with no meaning outside the database
<code>CHECK</code>	A constraint requiring a condition to hold on every row
Clause order	<code>FROM</code> , <code>WHERE</code> , <code>GROUP BY</code> , <code>HAVING</code> , <code>SELECT</code> , <code>ORDER BY</code> , <code>LIMIT</code>
<code>WHERE</code> against <code>HAVING</code>	Rows before grouping, groups after grouping
Anti-join	<code>LEFT JOIN</code> plus <code>WHERE right.id IS NULL</code> , to find non-matches
Three-valued logic	True, false and unknown. Ordinary comparisons with <code>NULL</code> give unknown, so use <code>IS NULL</code>
1NF	Every column holds a single value
2NF	1NF, and nothing depends on part of a composite key
3NF	2NF, and no non-key column depends on another non-key column
B-tree	The default index shape, ordered, serving equality, ranges and sorting
Covering index	Holds every column a query needs, so the table is never read
Leftmost prefix	A composite index serves filters starting from its first column
Selectivity	How much a condition narrows the result. Low selectivity defeats an index
Clustered index	The table stored inside the primary key's tree, as in InnoDB
Heap	An unordered table with all indexes pointing into it, as in Postgres
MVCC	Multiple row versions, so ordinary reads and writes do not block each other
Bloat	Dead row versions still occupying a Postgres table until vacuumed
Partitioning	Splitting one table into pieces inside one database
Sharding	Splitting data across separate database servers
Replication lag	How far behind a replica is, and why a fresh write may not be visible
Optimistic locking	No explicit lock first; a version column detects a conflict at write time
Pessimistic locking	A lock taken before the work, held until the transaction ends

Term	Meaning
CAP	During a network partition, choose linearizability or availability
N+1	One query for a list, then one more per item for an association

Symptom	Likely cause
A filter on a column returns zero rows unexpectedly	Comparing to <code>NULL</code> with <code>=</code> instead of <code>IS NULL</code>
A <code>NOT IN</code> query returns nothing at all	A null inside the subquery result
A <code>LEFT JOIN</code> losing its unmatched rows	A right-table condition placed in <code>WHERE</code>
A query slow only in production	Data volume, and a sequential scan the test data never triggered
An index that exists but is unused	Low selectivity, a function on the column, or a type conversion
Estimated rows far from actual rows	Stale statistics, so run <code>ANALYZE</code>
Writes slowing down over time	Too many indexes on the table
A Postgres table growing with no new rows	Dead versions from updates, and <code>VACUUM</code> held back by a long transaction
A row that disappears right after saving	Reading a replica before replication caught up
Requests blocking with the database idle	Connection pool exhausted, often by leaks or long transactions
Hundreds of near-identical queries per request	N+1 from a lazily loaded association
Intermittent failures under load, resolving on retry	Deadlock detection, or optimistic lock conflicts
One shard busy while the others idle	A shard key with uneven distribution